

توضیح پیاده‌سازی تسک Mansari Store Microservices

با سلام و احترام ،

پروژه مربوط به تسک میکروسرویس Book Store را تکمیل و ارسال کرده‌ام.

همچنین پروژه روی گیت هاب با ادرس زیر قابل دسترس می باشد:

https://github.com/Mansarix/mansari_microservice_store

در ادامه، خلاصه‌ای از وضعیت تطابق پروژه با نیازمندی‌های اعلام‌شده ارائه شده است:

1) Catalog Service با Redis Cache

در سرویس Catalog موارد زیر پیاده‌سازی شده است:

عملیات کامل CRUD برای موجودیت Book

پیاده‌سازی رفتار cache-first در متد GetBookById با استفاده از Redis

تنظیم مدت اعتبار کش روی ۵ دقیقه

حذف (invalidate) کش پس از عملیات update و delete

فایل‌های مرتبط:

```
src/Mansari.Store.Catalog/Mansari.Store.Catalog.Application/Book/Queries/GetBookById/GetBookByIdQueryHandler.cs
```

```
src/Mansari.Store.Catalog/Mansari.Store.Catalog.Infrastructure/Caching/RedisBookCacheService.cs
```

```
src/Mansari.Store.Catalog/Mansari.Store.Catalog.Application/Book/Command/UpdateBook/UpdateBookCommandHandler.cs
```

```
src/Mansari.Store.Catalog/Mansari.Store.Catalog.Application/Book/Command/DeleteBook/DeleteBookCommandHandler.cs
```

2) ایجاد سفارش با Eventual Consistency / Choreography Saga

فرآیند ایجاد سفارش به صورت غیرهمزمان و مبتنی بر integration event پیاده‌سازی شده است:

در CreateOrder، سفارش با وضعیت Pending ذخیره می‌شود

رویداد OrderCreatedEvent منتشر می‌شود

سرویس Catalog این رویداد را consume کرده و موجودی را بررسی می‌کند

در صورت کافی بودن موجودی، StockReservedEvent منتشر می‌شود

در صورت کافی نبودن موجودی، StockReservationFailedEvent منتشر می‌شود

سرویس Ordering رویداد نتیجه را دریافت کرده و وضعیت سفارش را به Confirmed یا Failed تغییر می‌دهد

فایل‌های مرتبط:

src/Mansari.Store.Contracts/Order/OrderCreatedEvent.cs

src/Mansari.Store.Contracts/Catalog/StockReservedEvent.cs

src/Mansari.Store.Contracts/Catalog/StockFailedEvent.cs

src/Mansari.Store.Ordering/Mansari.Store.Ordering.Application/Orders/Commands/CreateOrder/CreateOrderCommandHandler.cs

src/Mansari.Store.Catalog/Mansari.Store.Catalog.Infrastructure/Messaging/Consumers/OrderCreatedEventConsumer.cs

src/Mansari.Store.Ordering/Mansari.Store.Ordering.Infrastructure/Messaging/Consumers/StockReservedEventConsumer.cs

src/Mansari.Store.Ordering/Mansari.Store.Ordering.Infrastructure/Messaging/Consumers/StockReservationFailedEventConsumer.cs

3) Resiliency / Durability / Retry

برای تضمین پایداری و تحمل خطا، موارد زیر پیاده‌سازی شده است:

تعریف Exchange و Queue به صورت durable در RabbitMQ

انتشار پیام‌ها به صورت persistent

استفاده از الگوی Inbox / Outbox

استفاده از Polly برای retry در پردازش پس‌زمینه و مدیریت خطا

فایل‌های مرتبط:

src/Mansari.Store.Catalog/Mansari.Store.Catalog.Infrastructure/Messaging/Consumers/OrderCreatedEventConsumer.cs

src/Mansari.Store.Ordering/Mansari.Store.Ordering.Infrastructure/Messaging/Consumers/StockReservationFailedEventConsumer.cs

src/Mansari.Store.Catalog/Mansari.Store.Catalog.Infrastructure/Persistence/Outbox/OutboxProcessor.cs

src/Mansari.Store.Ordering/Mansari.Store.Ordering.Infrastructure/Persistence/Outbox/OutboxProcessor.cs

src/Mansari.Store.Catalog/Mansari.Store.Catalog.Infrastructure/Persistence/Inbox/InboxMessage.cs

src/Mansari.Store.Ordering/Mansari.Store.Ordering.Infrastructure/Persistence/Inbox/InboxMessage.cs

در صورت Down بودن یکی از سرویس‌ها، پیام‌ها در RabbitMQ باقی می‌مانند و پس از در دسترس قرار گرفتن سرویس مربوطه پردازش می‌شوند.

4) استفاده از MediatR

برای پیاده‌سازی use case اصلی از MediatR استفاده شده است، از جمله:

CreateBook

CreateOrder

Queryهایی مانند GetBookById

فایل‌های مرتبط:

src/Mansari.Store.Catalog/Mansari.Store.Catalog.Application/Book/Command/CreateBook/Command/CreateBookCommandHandler.cs

src/Mansari.Store.Ordering/Mansari.Store.Ordering.Application/Orders/Commands/CreateOrder/Command/CreateOrderCommandHandler.cs

src/Mansari.Store.Catalog/Mansari.Store.Catalog.Application/Book/Queries/GetBookById/Query/GetBookByIdQueryHandler.cs

5) Clean Architecture

هر دو سرویس Catalog و Ordering با ساختار Clean Architecture پیاده‌سازی شده‌اند و شامل لایه‌های زیر هستند:

API, Application, Domain, Infrastructure

در لایه اینفرا :

- برای مدیریت بهتر و تمیزتر ، از ویلیو ابجکت ها استفاده شده
- دامین اکسپشن برای تمایز اکسپشن ها لایه دامین از سایر لایه ها
- بیس کلاس ها برای راحت تر کردن فرایند اضافه کردن موارد جدید آینده
- همه انتیتی ها از نوع ریچ دامین هستند و ولیدیشن ، مدیریت و متدها و ... را در خود پیاده سازی کرده اند

6) Dockerization

پروژه به صورت کامل containerized شده است:

Dockerfile جداگانه برای هر سرویس

فایل docker-compose.yml در ریشه پروژه شامل:

Catalog API

Ordering API

Gateway

Redis

RabbitMQ

PostgreSQL برای هر سرویس

فایل های مرتبط:

docker-compose.yml

src/Mansari.Store.Catalog/Mansari.Store.Catalog.Api/Dockerfile

src/Mansari.Store.Ordering/Mansari.Store.Ordering.Api/Dockerfile

src/Mansari.Store.Gateway/Dockerfile

در صورت نیاز، خوشحال می شوم توضیح تکمیلی درباره جریان پیام ها یا تصمیمات معماری اتخاذشده ارائه دهم.

با احترام،

محمود انصاری

09301761527